

1. Przygotowanie
 - a. środowisko: <https://a.lp-lab.cloud:8006>
 - b. repozytorium: <https://github.com/gwiatu-lp/cri.git>
 - c. logowanie do stacji i podział środowisk
 - i. terminal
 - ii. wprowadzenie do środowisk
2. Podman
 - a. Praktyczne zastosowanie
 - i. Podman może być stosowany w różnych scenariuszach, takich jak:
 1. Uruchamianie aplikacji w izolowanych kontenerach.
 2. Tworzenie lekkich środowisk testowych.
 3. Automatyzacja procesów CI/CD.
 4. Użycie w systemach z podwyższonymi wymaganiami bezpieczeństwa.
 5. Uproszczenie procesu aktualizacji OS
 - b. Przegląd zagadnień związanych z Podman
 - i. Podman to narzędzie do zarządzania kontenerami, które jest alternatywą dla Dockera. Główne różnice między Podmanem a Dockerem to:
 1. Bezpieczeństwo: Podman nie wymaga działania usługi daemon i może być używany bez uprawnień administratora (rootless containers).
 2. Kompatybilność: Obsługuje obrazy i polecenia zgodne z Dockerem.
 3. Podman vs Docker:
 - a. Docker działa w modelu klient-serwer, a Podman działa jako samodzielne narzędzie.
 - b. Podman pozwala na uruchamianie kontenerów bez potrzeby działania centralnego daemona.
 4. Instalacja:

sudo dnf install podman -y
 5. Modyfikacja pliku /etc/containers/registries.conf

sudo vi /etc/containers/registries.conf
 6. Dodanie parametrów z pliku ~/cri/registries.conf

unqualified-search-registries = ['docker.io']

[[registry]]
location="docker.io"
username="tovali3200"
password="a82a8222-4e29-4ec1-af22-2bcde01e965a"

- c. Tworzenie obrazów przy użyciu narzędzia Podman
 - i. Podobnie jak Docker, Podman pozwala na budowanie obrazów kontenerów z użyciem Dockerfile. Proces obejmuje:
 1. Definiowanie warstw obrazu w Dockerfile.
 2. Użycie narzędzia podman build do stworzenia obrazu.
 3. Weryfikację stworzonego obrazu za pomocą podman images.
- d. Tworzenie obrazów przy użyciu podman build (cri/workshop/lab-1)
 - i. Dokumentacja: <https://docs.docker.com/reference/dockerfile/>
 - ii. Składnia polecenia:


```
podman build -t debian-mysql \
--build-arg MYSQL_ROOT_PASSWORD=rootpassword \
--build-arg MYSQL_DATABASE=developer_db \
--build-arg MYSQL_USER=developer \
--build-arg MYSQL_PASSWORD=developerpassword \
.
```
 - iii. Opcje dodatkowe:
 - a. --file – określenie alternatywnego pliku Dockerfile.
 - b. --layers – wykorzystanie istniejących warstw w celu przyspieszenia procesu budowy
- e. Uruchamianie kontenerów za pomocą Podman: (cri/workshop/lab-2)
 - i. Uruchomienie prostego kontenera: podman run -d --name kontener-example obraz
 - ii. Uruchomienie kontenera z interaktywnym dostępem: podman run -it --rm obraz bash
 - iii. Uruchomienie kontenera z mapowaniem portów: podman run -d -p 8080:80 --name kontener-web serwer-www
 - iv. Uruchomienie kontenera z woluminem do przechowywania danych: podman run -d -v /ścieżka-lokalna:/ścieżka-kontenerowa --name kontener-z-danymi obraz
 - v. Uruchomienie kontenera dla konkretnego adresu IP: podman run -d --ip 192.168.1.100 -p 8080:80 --name kontener-ip obraz
 - vi. Wyświetlenie działających kontenerów: podman ps
 - vii. Wyświetlanie wszystkich kontenerów (w tym zatrzymanych): podman ps -a
 - viii. Zatrzymanie kontenera: podman stop kontener-example
 - ix. Usunięcie kontenera: podman rm kontener-example

Lista komend:

```
#listowanie kontenerow
podman ps

#listowanie obrazow
podman images
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

```
#start kontenera
podman start <container_name_or_id>

#zatrzymanie kontenera
podman stop <container_name_or_id>

#usuniecie kontenera
podman rm <container_name_or_id>

#usuniecie obrazu
podman rmi <image_name_or_id>

#utworzenie nowego kontenera z obrazu
podman run -d <image_name_or_id>

#generowanie pliku yaml
podman generate kube

#wczytanie manifestu
podman play kube - <<EOF
EOF
```

- f. Zarządzanie obrazami: push, pull i tagowanie (cri/workshop/lab-3)
 - i. Wyszukiwanie obrazów: **sudo podman search**
 - ii. Logowanie do repozytorium:
**podman login -u admin -p student **
harbor.apps.prod0.lab.linuxpolska.pl
 - iii. Pobieranie obrazów: **podman pull <obraz>**
 - iv. Wysyłanie obrazów do rejestru: **podman push obraz**
registry.example.com/uzytkownik/obraz
 - v. Tagowanie obrazów: **podman tag obraz nowa-nazwa**
- g. Zarządzanie sieciami w podman (cri/workshop/lab-4)
 - i. omówienie sieci
 - ii. tworzenie sieci: **podman network create --subnet=[xxx.xxx.xxx.xxx/24](#)**
--gateway=[xxx.xxx.xxx.xxx](#)
 - iii. listowanie podsieci: **podman network ls**
 - iv. informacje o podsieci: **podman network inspect**
- h. Przekształcenie workflow Docker do Podman
 - i. Większość poleceń Dockera ma swoje odpowiedniki w Podmanie.
 - ii. Można łatwo zastąpić docker przez podman w skryptach.
 - iii. Podman posiada aliasy umożliwiające użytkownikom przyzwyczajonym do Dockera łatwe przejście: **alias docker=podman**
 - iv. Narzędzie **podman system migrate** pomaga w migracji istniejących konfiguracji Dockera do Podmana.

3. Instalacja klastra Kubernetes:

- a. W celu instalacji klastra Kubernetes wykorzystamy narzędzie kubeadm.
- b. Minimalne wymagania dotyczące node'a klastra to co najmniej dwa node'y o następujących parametrach:
 - OS: Ubuntu
 - RAM: 2GB lub więcej
 - CPU: 2 CPU lub więcej dla node'a control-plane
 - Połączenie sieciowe pomiędzy nodami
- c. Przed zainstalowaniem klastra niezbędne jest przygotowanie runtime kontenerów oraz narzędzie kubetools: kubeadm, kubeinit, kube-proxy oraz kubectl.
- d. Runtime dla kontenerów jest niezbędny aby móc korzystać z kontenerów
- e. Kubernetes wspiera różnorodne środowiska runtime dla kontenerów:
 - containerd
 - CRI-O
 - Docker Engine
 - Mirantis Container Runtime
- f. Poniżej znajduje się link do pobrania repozytorium, z którego pobrać można skrypt instalacyjny środowiska runtime na Wasze stacje robocze:

<https://github.com/qwiatu-lp/cri.git>

[setup-container.sh](#)

- g. Przed instalacją klastra skorzystamy ze skryptu instalacyjnego dla narzędzi Kubernetes, który zawiera
 - kubeadm**: wykorzystamy go do instalacji i zarządzania naszym klastrem Kubernetes
 - kubelet**: core service, który uruchamia pody
 - kubectl**: klient/interfejs, którym będziemy wykonywać polecenia do uruchamiania i zarządzania aplikacjami Kubernetes

<https://github.com/qwiatu-lp/cri.git>

[setup-kubetools.sh](#)

h.

- Przed skonfigurowaniem klastra warto więc dowiedzieć się nieco o wymaganiach sieciowych dla różnych węzłów.
W Kubernetes wykorzystywane są różne rodzaje komunikacji sieciowej.

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

Pierwszym z nich jest komunikacja między węzłami.

Jest to część obsługiwana przez sieć fizyczną, w której wszystkie węzły są ze sobą bezpośrednio połączone.

Istnieje komunikacja External-to-Service, która jest obsługiwana przez zasoby Kubernetes Service.

Istnieje komunikacja Pod-to-Service, która również jest obsługiwana przez usługi Kubernetes.

Komunikacja Pod-to-Pod jest obsługiwana przez wtyczkę sieciową.

I wreszcie, istnieje komunikacja kontener-kontener, która jest stosowana tylko wtedy, gdy wiele kontenerów działa w tym samym Pod.

Komunikacja między tymi kontenerami jest obsługiwana w ramach samego Pod.

Ważną częścią konfiguracji klastra Kubernetes jest dodatek sieciowy, ponieważ bez dodatku sieciowego nie można zdefiniować sieci Pod.

Obecnie dodatek sieciowy jest dostarczany przez ekosystem i dostępne są różne dodatki sieciowe.

Problematiczne jest to, że sam Vanilla Kubernetes nie jest dostarczany z domyślnym dodatkiem, a to dlatego, że Cloud Native Computing Foundation nie chce faworyzować jednego konkretnego rozwiązania

Oznacza to, że po uruchomieniu kubeadm należy samodzielnie skonfigurować dodatek sieciowy.

Kubernetes udostępnia Container Network Interface, czyli CNI, jako ogólny interfejs, który umożliwia łatwe korzystanie z różnych wtyczek.

Dostępność konkretnych funkcji zależy od używanej wtyczki sieciowej.

Należy na przykład szukać obsługi Networkpolicy, IPv6 lub Role Based Access Control.

Dostępnych jest wiele dodatków sieciowych. Chcę tylko wspomnieć o czterech najważniejszych.

Cilium jest tym, którego będziemy używać w tym szkoleniu.

Najpopularniejszym rozwiązaniem jest Calico.

Istnieje Flannel, który jest ogólnym dodatkiem sieciowym, który był często używany w przeszłości, ale nie obsługuje polityki sieciowej.

Istnieje Multus, który jest wtyczką, która może współpracować z wieloma

wtyczkami sieciowymi i jest obecnie domyślną wtyczką w OpenShift, która jest bardzo ważną dystrybucją Kubernetes.

Jest też Weave, wspólny dodatek sieciowy, który obsługuje również typowe funkcje.

i.

W dalszej części tej szkolenia pokażę, jak zainstalować klaster przy użyciu kubeadm.

Zanim to zrobimy, dobrze jest wiedzieć trochę o tym, co się wydarzy.

Tak więc, podczas uruchamiania kubeadm init, wykonywane są różne fazy.

Zaczyna się od preflight.

Zapewnia to spełnienie wszystkich warunków i pobranie podstawowych obrazów kontenerów.

Następnie zajmuje się certyfikatami, generowany jest samopodpisany Kubernetes Certificate Authority i tworzone są powiązane certyfikaty dla apiserver, etcd i proxy.

Kubernetes często korzysta z tych certyfikatów PKI i są one bardzo ważne. Następnie znajduje się kubeconfig, w którym konfigurowane są pliki konfiguracyjne dla podstawowych usług Kubernetes.

Następnie można uruchomić kubelet.

Gdy kubelet jest już dostępny, konfigurowany jest control-plane.

W tym celu kubelet używa statycznych manifestów pod. Tworzy je i uruchamia dla apiservera, menedżera kontrolera i schedulera.

W tym momencie używany jest etcd. Tworzone i uruchamiane są więc statyczne manifesty Pod dla etcd.

Następnie procedura jest kontynuowana wraz z przesyłaniem konfiguracji. ConfigMaps są tworzone dla ClusterConfiguration i konfiguracji komponentów kubelet.

Upload-certs jest miejscem, w którym wszystkie certyfikaty są przesyłane do /etc/kubernetes/pki na węzle control-plane.

Następnie węzeł jest oznaczany jako control plane i generowany jest token bootstrap.

Jest to token, którego można użyć do dołączenia do innych węzłów.

Następnym krokiem jest kubelet-finalized, który finalizuje ustawienia kubeleta.

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Gdy to zrobimy, pozostaje już tylko jeden krok do wykonania, a jest nim dodatek. W tej części instalowane są dodatki coredns i kube-proxy.

Po wykonaniu wszystkich tych czynności mamy już działające środowisko klastra Kubernetes.

Teraz ważne jest, abyście wiedzieli trochę o tych różnych fazach, ponieważ w przypadku, gdy coś pójdzie nie tak, warto wiedzieć, gdzie coś poszło źle i jak wpłynie to na wszystko inne, co może być przydatne do rozwiązywania problemów.

- j. Instalacja klastra - pobranie oraz uruchomienie skryptów:

```
# Sklonowanie repozytorium ze skryptami:
```

```
git clone https://github.com/qwiatu-lp/cri.git
```

```
# Instalacja CRI:
```

```
sudo bash ~/cri/setup-container.sh
```

```
# Instalacja kubetools:
```

```
sudo bash ~/cri/setup-kubetools.sh
```

Instalacja klastra - przygotowanie pliku kubeadm-config.yaml z konfiguracją:

```
vim ~/cri/kubeadm-config.yaml
```

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: stable
apiServer:
  certSANs:
    - "k8s-X-master-1.k8s.lp-lab.cloud"
controlPlaneEndpoint: "k8s-X-master-1.k8s.lp-lab.cloud:6443"
```

Instalacja klastra:

```
sudo kubeadm init --config ~/cri/kubeadm-config.yaml
```

Przygotowanie klienta "kubectl":

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

sudo apt-get install bash-completion
kubectl completion bash | sudo tee /etc/bash_completion.d/kubectl >
/dev/null

echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc

sudo su -l $USER
```

k. Instalacja wtyczki sieciowej Cilium:

i.

```
sudo snap install helm --classic

kubectl -n kube-system delete ds kube-proxy

kubectl -n kube-system delete cm kube-proxy

helm repo add cilium https://helm.cilium.io/

helm upgrade --install cilium cilium/cilium --version 1.16.4 \
--namespace kube-system \
--set kubeProxyReplacement=true \
--set k8sServiceHost=$(hostname).k8s.lp-lab.cloud \
--set k8sServicePort=6443 \
--set hubble.relay.enabled=true \
--set hubble.ui.enabled=true
```

ii.

l. Operacje z **kubeadm init**:

- i. --apiserver-advertise-address
- ii. --config
- iii. --dry-run
- iv. --pod-network-cidr
- v. --service-cidr

m. Dodawanie node-ów do klastra:

- i. W przypadku utraty join token'u lub jego wygaśnięcia używamy polecenia (wykonać na k8s-X-master-1):

```
sudo kubeadm token create --print-join-command
```


- ii. Na pozostałych nodach należy wkleić **wynik** powyższego polecenia poprzedzając poleceniem SUDO. PRZYKŁAD: **sudo kubeadm join k8s-X-master-1.k8s.lp-lab.cloud:6443 \ --token YYY --discovery-token-ca-cert-hash sha256:ZZZ**
- n. Zarządzanie kontekstami:
 - i. **kubectl config view**
 - ii. **kubectl config get-contexts**
 - iii. **kubectl config set-context**
- 4. Czym są manifesty / zasoby ?
 - a. **kubectl api-resources**
- 5. Używanie namespace-ów (~/cri/workshop/k8s-lab-1)
 - a. obiekt wykorzystywany do grupowania wielu aplikacji w przestrzenie
 - i. tworzenie namespace za pomocą podejścia imperatywnego
 - 1. **kubectl create ns mojaapka**
- 6. Używanie deployment'ów (~/cri/workshop/k8s-lab-2):
 - a. Deployment to podstawowa opcja uruchamiania kontenerów na Kubernetes
 - b. Deployment jest odpowiedzialny za uruchamianie i skalowanie podów
 - c. Deployment używa ReplicaSet aby zarządzać skalowalnością
 - d. Deployment oferuje politykę wdrażania RollingUpdate, aby uzyskać bezprzerwową funkcjonalność aktualizacji aplikacji.
 - e. Uruchomienie deploymentu za pomocą tzw. podejścia imperatywnego:
 - i. **kubectl create deployment -h**
 - ii. **kubectl create deployment deploy-www --image nginx --replicas=3**
 - iii. **kubectl get po -w**
 - iv. **kubectl get all**
- 7. Używanie DaemonSet:
 - a. DaemonSets są powszechnie używane do uruchamiania agentów. Jest to zasób, który uruchamia jedną instancję aplikacji na każdym węźle klastra. Z tego powodu nie ma liczby replik zdefiniowanych w DaemonSet. Po prostu uruchamia jednego agenta na każdym węźle klastra.
 - b. Sprawdźmy czy mamy jakieś obiekty tego typu w klastrze:
 - i. **kubectl get ds -A**
 - c. Powoływanie obiektu DaemonSet dla obrazu nginx:
 - i. **kubectl create deploy daemonset-www --image=nginx --dry-run=client -o yaml > mydaemon.yaml**
 - ii. Podnosimy do edycji:
 - vi mydaemon.yaml
 - zmieniamy:
 - Deployment > DaemonSet
 - i usuwamy:
 - replicas:1
 - strategy: {}
 - iii. **kubectl apply -f mydaemon.yaml**
 - iv. **kubectl get ds -n default**

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

8. Używanie StatefulSet:

- a. StatefulSet oferuje kilka funkcji, które są potrzebne w aplikacjach stanowych.
 - Zapewnia gwarancje dotyczące kolejności i unikalności podów
 - Utrzymuje stały identyfikator dla każdego z podów, które tworzy, a pody w StatefulSet nie są wymienne.
 - Każdy Pod ma trwały identyfikator, który utrzymuje podczas zmiany harmonogramu.
 - Unikalne identyfikatory podów ułatwiają dopasowanie istniejących woluminów do zastąpionych podów.

- b. Spójrzmy na przykładowy plik **statefuldemo.yaml**:

```
cat statefuldemo.yaml
```

```
kubectl apply -f statefuldemo.yaml
```

```
kubectl get statefulset lub kubectl get sts
```

- c. Rozwiązanie problemu ze storage

9. Uruchamianie pojedynczych podów:

- a. Główne ryzyka:
 - i. Brak ochrony zrównoważonego obciążenia
 - ii. Brak LoadBalancing'u
 - iii. Brak bezprzerwowej aktualizacji aplikacji
- b. Stosujemy pojedyncze pody wyłącznie w celu testowania, rozwiązywania problemów lub analizy aplikacji/klastra.
- c. W każdym innym razie stosujemy obiekty tj. Deployment, DaemonSet lub Statefulset
- d. Uruchomienie przykładowego pojedynczego pod'a:
 - i. **kubectl run pod -h | less**
 - ii. **kubectl run pod-with-sleep --image=busybox -- sleep 3600**
 - iii. **kubectl get po**

10. Zarządzanie inicjalizacją podów:

- a. Jeśli więc wymagane jest przygotowanie przed uruchomieniem głównego kontenera, odpowiedzią jest init container. Init container'y są częścią pod'a i są uruchamiane razem z głównym kontenerem. Init container działa do końca, a po jego zakończeniu można uruchomić główny kontener. Init container'y są używane w każdym przypadku, w którym wymagana jest wstępna konfiguracja aplikacji.
- b. Przykładowy plik:
 - i. **kubectl apply -f initme.yaml**
 - ii. Modyfikacja parametrów na sleep 30

```
apiVersion: v1
kind: Pod
metadata:
  name: init-demo-sleep
spec:
  containers:
  - name: nginx
    image: nginx
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

```
ports:
- containerPort: 80
initContainers:
- name: install
  image: busybox
  command:
  - "sleep"
  - "30"
```

iii. **kubectl apply -f initme.yaml**

11. Skalowanie aplikacji:

- a. Używamy polecenia **kubectl scale** w stosunku do obiektów tj. Deployment, ReplicaSet lub StatefulSet:
 - i. **kubectl scale deployment nginxsvc --replicas=3**
- b. Użycie komendy **kubectl get all** z parametrem **--selector**
 - i. **kubectl get all --selector app=deploy-www**
- c. Alternatywą jest użycie obiektu: HorizontalPodAutoscaler.

12. Używanie Sidecar Containers:

- a. W niektórych przypadkach potrzebny jest jednak dodatkowy kontener do modyfikacji lub prezentacji danych wygenerowanych przez główny kontener. Zdefiniowano też pewne szczególne przypadki użycia, którymi są:
 - i. sidecar, który zapewnia dodatkową funkcjonalność głównemu kontenerowi
 - ii. ambassador, który jest używany jako proxy do zewnętrznego łączenia kontenerów
 - iii. adapter, który służy do standaryzacji lub normalizacji danych wyjściowych głównego kontenera.
- b. Przykład w oparciu o plik **sidecarlog.yaml**
 - i. **kubectl apply -f sidecarlog.yaml**
 - ii. **kubectl exec -it two-containers -c nginx-container -- cat /usr/share/nginx/html/index.html**

13. Zarządzanie storage:

- a. Dokumentacja:

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/#types-of-persistent-volumes>
- b. Trwałe woluminy są specyficzne dla każdego z namespace'ów, więc to administrator namespace'a musi upewnić się, że trwałe woluminy są dostępne.
- c. W jaki sposób zamierzasz połączyć się z poda do trwałego woluminu? Zamierzamy to zrobić za pomocą PVC, co jest skrótem od persistent volume claim. To żądanie trwałego wolumenu jest niezależnym zasobem API, który zostanie skonfigurowany jako wolumen pod'a. Wystarczy więc ustawić typ woluminu pod na persistence volume claim.
- d. Żądanie trwałego wolumenu to po prostu żądanie pamięci masowej. Żądanie to jest wyrażone jako rozmiar i to, czy pamięć masowa powinna być zapisywalna (tzw. pamięć RWX) czy tylko do odczytu (tzw. pamięć RWO). Jak

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

to działa? Działa to w ten sposób, że uruchamiasz swój pod i PVC w określonym namespace, a następnie PVC sprawdzi w tym namespace, czy dostępna jest określona pamięć masowa. Jeśli istnieje pasujący trwały wolumen, to jest to proste, a PVC zostanie powiązane z pasującym PV. Istnieje jednak opcja elastyczności, która jest realizowana za pomocą StorageClass. StorageClass jest używany do automatycznego dostarczania pamięci masowej, jeśli nie jest ona dostępna w momencie nadejścia żądania PVC. Aby móc pracować z obiektami StorageClass, potrzebny jest również odpowiedni provider dla trwałego woluminu (PV).

- e. Czym jest PV Provisioner (CSI driver)? Dostawca pamięci masowej komunikuje się z pamięcią masową dostępną w określonym namespace.
- f. StorageClass to obiekt API reprezentujący Provisioner'a. Tak więc bez względu na to, jaką masz pamięć masową, tak długo, jak masz provisioner'a i StorageClass która z nim współpracuje, StorageClass może przekazywać pamięć masową na żądanie PVC.
- g. Tak więc w przypadku, gdy PVC zgłasza żądanie pamięci masowej i nie ma dostępnej pamięci masowej, klasa pamięci masowej przechwyci to żądanie i stworzy trwały wolumin (PV) na żądanie zgodnie z wymaganiami określonymi w PVC. Dzięki temu Kubernetes może dynamicznie przydzielać przestrzeń dyskową, gdy jest ona potrzebna.
- h. Konfiguracja PersistenceVolumes:
 - i. Przykład:
 - 1. **vi pv.yaml**
 - 2. **kubectl apply -f pv.yaml**
 - i. Konfiguracja PersistenceVolumeClaim:
 - i. Przykład:
 - 1. **vi pvc.yaml**
 - 2. **kubectl apply -f pvc.yaml**
 - 3. Troubleshooting
 - j. Konfiguracja StorageClass:
 - i. Instalacja serwera NFS na node'dzie - "**k8s-X-master-1**":

```
# na nodzie control-plane:

sudo apt install nfs-server -y

sudo mkdir /nfsexport

sudo bash -c \
'echo "/nfsexport *(rw,no_root_squash)" > /etc/exports'

sudo systemctl enable nfs-server
sudo systemctl restart nfs-server
```

1.

- ii. Instalacja klienta NFS na node'ach - "k8s-X-worker-2" i "k8s-X-worker-3":

```
sudo apt install nfs-client -y
showmount -e k8s-X-master-1.k8s.lp-lab.cloud
```

- iii. Instalacja dodatku nfs-subdir-external-provisioner na węźle "k8s-X-master-1":

```
helm repo add nfs-subdir-external-provisioner \
https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner

helm install nfs-subdir-external-provisioner \
nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
--set nfs.server=k8s-X-master-1.k8s.lp-lab.cloud \
--set nfs.path=/nfsexport
```

- iv. Zarządzanie PVC:

1. **kubectl get pv**
2. **kubectl apply -f nfs-provisioner-pvc-test.yaml**
3. **kubectl get pv,pvc**

- v. Ćwiczenia z PVC i SC:

1. **kubectl apply -f another-pvc-test.yaml**
2. **kubectl get pvc**
3. **kubectl patch storageclass nfs-client -p '{"metadata": {"annotations":{"[storageclass.kubernetes.io/is-default-class](https://kubernetes.io/docs/concepts/configuration/configmap/)":"true"}}}'**
4. **kubectl get pvc**
5. [Przykład](#)

- k. Konfiguracja ConfigMap i Secret'ów (~/cri/workshop/k8s-lab-3)

- i. dokumentacja: <https://kubernetes.io/docs/concepts/configuration/configmap/>
- ii. ConfigMap jest zasobem API używanym do przechowywania danych specyficznych dla witryny. Secret to ConfigMap zakodowany w base64.
- iii. Nie ma żadnej fundamentalnej różnicy między Secret i ConfigMap, z jedynym wyjątkiem, że Secret są trudniejsze do odczytania, ponieważ najpierw trzeba je zdekodować. ConfigMaps są używane do przechowywania zmiennych środowiskowych, parametrów startowych lub plików konfiguracyjnych. Gdy plik konfiguracyjny jest używany w ConfigMap lub Secret, jest montowany jako wolumin, aby zapewnić dostęp do jego zawartości.

14. Zarządzanie dostępem do aplikacji (CNI, Services):

- a. Omówienie CNI:

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

- i. Calico
- ii. Flannel
- iii. Cilium
- b. Omówienie usług (services) (~/cri/workshop/k8s-lab-4):
 - i. [NodePort](#)
 - ii. ClusterIP
 - iii. [LoadBalancer](#)
 - iv. HostNetwork (konfiguracja w deploy/ds/sts)
 - v. [ExternalName](#)
 - vi. [Headless](#)
- c. Konfiguracja Ingress:
 - i. Instalacja:

```
helm upgrade --install ingress-nginx ingress-nginx \
--repo https://kubernetes.github.io/ingress-nginx \
--namespace ingress-nginx \
--create-namespace \
--set controller.hostNetwork=true \
--set controller.kind=DaemonSet

kubecti -n ingress-nginx get pods
```

- d. Uruchomienie przykładowej aplikacji
 - i. Przykładowy deployment

```
kubectl create deploy nginxsvc --image=nginx --port=80

kubectl expose deploy nginxsvc
```

- ii. Używanie port forwarding'u do bezpośredniego dostępu do aplikacji

```
kubectl create deploy nginxsvc --image=nginx --port=80

kubectl expose deploy nginxsvc

kubectl create ingress nginxsvc --class=nginx \
--rule=nginxsvc.info/*=nginxsvc:80

kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller \
8080:80 &

curl -H 'Host: nginxsvc.info' http://localhost:8080
```

```
# stop port-forward
# kill %1
```

iii. Wsparcie dla ruchu HTTP

```
kubectl create ingress nginxsvc \
--rule=nginxsvc.appsX.k8s.lp-lab.cloud/*=nginxsvc:80

curl -kL http://nginxsvc.appsX.k8s.lp-lab.cloud
```

iv. Wsparcie ruchu HTTPS

```
kubectl create ingress nginxsvctls --class=nginx \
--rule=nginxsvc.appsX.k8s.lp-lab.cloud/*=nginxsvc:80,tls

curl -kL https://nginxsvc.appsX.k8s.lp-lab.cloud
```

v. Konfiguracja i omówienie IngressClass

1. **kubectl get ingressclass -o yaml**

vi. Przykłady:

1. Przykład z domyślną stroną web-apps:

```
kubectl create deploy web1 --image=nginx --port=80
kubectl expose deploy web1 \
--port 8080 --target-port 80

kubectl create deploy web2 --image=httpd --port=80
kubectl expose deploy web2 \
--port 8080 --target-port 80

kubectl exec deploy/web2 -- \
ls /usr/local/apache2/htdocs/

kubectl exec deploy/web2 -- \
mkdir /usr/local/apache2/htdocs/web2

kubectl exec deploy/web2 -- \
cp /usr/local/apache2/htdocs/index.html \
/usr/local/apache2/htdocs/web2/index.html
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

```
kubectl create ingress web-ingress --class=nginx \
--rule="/*=web1:8080" \
--rule="/web2/*=web2:8080" \
--dry-run=client -o yaml | kubectl apply -f -

kubectl describe ingress/web-ingress

curl -L http://any.appsX.k8s.lp-lab.cloud

curl -L http://any.appsX.k8s.lp-lab.cloud/web2
```

2. Różne virtual host'y:

```
kubectl create ingress newweb-ingress --class=nginx \
--rule="web1.appsX.k8s.lp-lab.cloud/*=web1:8080" \
--rule="web2.appsX.k8s.lp-lab.cloud/*=web2:8080" \
--dry-run=client -o yaml | kubectl apply -f -
```

15. Zarządzanie klastrem:

- a. Analiza node'ów klastra
- b. Używanie **crictrl** do zarządzania i analizowania kontenerami na węźle
- c. Uruchomienie tzw. static pod
 - i. **ssh k8svc@k8s-X-worker2**
 - ii. **sudo vi /etc/kubernetes/manifests/staticpod.yaml**
- d. Zarządzanie stanem node'ów
 - i. Drain
 1. **kubectl drain --ignore-daemonsets --delete-emptydir-data**
 - ii. Cordon
 1. **kubectl cordon <node>**
 2. **kubectl describe node <node>**
 3. **kubectl get nodes**
 - iii. Uncordon
 1. **kubectl uncordon <node>**
- e. Zarządzanie usługami node'a
 - i. **ps aux | grep kubelet**
 - ii. **ps aux | grep containerd**
 - iii. **systemctl status kubelet**
 - iv. **sudo systemctl stop kubelet**
 - v. **sudo systemctl start kubelet**

16. Przeprowadzanie zadań konserwacyjnych na node'ach

- a. Instalacja i przegląd funkcji metrics-server
 - i. dokumentacja: <https://github.com/kubernetes-sigs/metrics-server.git>
 - ii. instalacja

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.


```
kubectl apply -f \
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

kubectl -n kube-system get pods -l k8s-app=metrics-server
```

- iii. modyfikacja
 - 1. **kubectl -n kube-system edit deployment metrics-server**
 - 2. W spec.template.spec.containers.args, zmień na następujące wartości:
 - a. **--kubelet-insecure-tls**
 - b. **--kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname**
- iv. **kubectl -n kube-system logs metrics-server<TAB>** powinna pokazać "Generating self-signed cert" i "Serving securely on [::]:443"
- v. **kubectl top pods --all-namespaces** zwróci najbardziej aktywne pody
- b. Backup bazy danych etcd. Wykonać **k8svc@k8s-X-master1**

```
sudo apt install etcd-client

sudo etcdctl --help; sudo ETCDCTL_API=3 etcdctl --help

ps aux | grep etcd

sudo ETCDCTL_API=3 etcdctl --endpoints=localhost:2379 \
--cacert /etc/kubernetes/pki/etcd/ca.crt \
--cert /etc/kubernetes/pki/etcd/server.crt \
--key /etc/kubernetes/pki/etcd/server.key \
get / --prefix --keys-only

sudo ETCDCTL_API=3 etcdctl --endpoints=localhost:2379 \
--cacert /etc/kubernetes/pki/etcd/ca.crt \
--cert /etc/kubernetes/pki/etcd/server.crt \
--key /etc/kubernetes/pki/etcd/server.key \
snapshot save /mnt/etcdbackup.db

sudo mkdir backup-manifests
sudo cp -r /etc/kubernetes/manifests/* backup-manifests/
```

- c. Przywracanie kopii bazy danych etcd

```
sudo ETCDCTL_API=3 etcdctl --endpoints=localhost:2379 \
--cacert /etc/kubernetes/pki/etcd/ca.crt \
--cert /etc/kubernetes/pki/etcd/server.crt \
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

```
--key /etc/kubernetes/pki/etcd/server.key \
snapshot restore /mnt/etcdbackup.db --data-dir /var/lib/etcd-restore

sudo rm -rf /etc/kubernetes/manifests/kube-controller-manager.yaml
sudo rm -rf /etc/kubernetes/manifests/kube-apiserver.yaml
sudo rm -rf /etc/kubernetes/manifests/kube-scheduler.yaml

sudo mv /etc/kubernetes/manifests/etcd.yaml etcd.yaml
sudo vi etcd.yaml
sudo mv etcd.yaml /etc/kubernetes/manifests/

sudo cp backup-manifests/kube-apiserver.yaml \
/etc/kubernetes/manifests/

sudo cp backup-manifests/kube-controller-manager.yaml \
/etc/kubernetes/manifests/

sudo cp backup-manifests/kube-scheduler.yaml \
/etc/kubernetes/manifests/
```

- d. Przeprowadzanie aktualizacji węzłów
 - i. <https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>
 - e. Klaster o wysokiej dostępności - omówienie
17. Zarządzanie scheduler'em
- a. Kube-scheduler bierze na siebie zadania związane z znalezieniem node'a w celu zaschedulowania nowych podów
 - b. Node'y mogą być filtrowane wg specyficznych wymagań tj.:
 - i. Wymagania co do zasobów sprzętowych
 - ii. Reguł affinity i anti-affinity
 - iii. Taints i Tolerations
 - iv. Scheduler na podstawie wprowadzonych danych wybiera node'y nadając im punktację a następnie wybiera najwyżej ocenionego node'a w celu zaschedulowania pod'a zgodnego z wymaganiami
 - c. Kubelet zajmuje się pobraniem obrazu i instalacją pod'a na wskazanych przez kube-scheduler'a node'dzie
 - d. Ustawianie preferencji node'a
 - i. `kubectl label node`
 - ii. [nodeSelector](#)
 - iii. [nodeName](#)
 - e. Ustawienie affinity i anti-affinity
 - i. dokumentacja: <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>
 - ii. **pod-with-node-affinity.yaml**
 - iii. **pod-with-node-anti-affinity.yaml**
 - iv. **pod-with-pod-affinity.yaml**

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

1. requiredDuringSchedulingIgnoredDuringExecution = warunek musi być spełniony
 2. preferredDuringSchedulingIgnoredDuringExecution = warunek nie musi być spełniony
- f. Zarządzanie taints i tolerations
- i. dokumentacja: <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>
 - ii. Przykłady:
 1. **taint-tolerations.yaml**
 2. **taint-tolerations2.yaml**
- g. Omówienie LimitRange i Quota.
- h. Konfiguracja Quota
- i. doc: <https://kubernetes.io/docs/concepts/policy/resource-quotas/#quota-scopes>
 - ii. `kubectl create quota default --hard=`
- i. Konfiguracja LimitRange:
- i. Przykład:
 1. **limitrange.yaml**
18. Sieć w Kubernetes:
- a. Omówienie sieci CNI i plugin'ów sieciowych
 - b. Polityki networkPolicy do zarządzania ruchem pomiędzy podami
 - c. Konfiguracja networkPolicy w klastrze
19. Zarządzanie bezpieczeństwem klastra
- a. Omówienie dostępu do API klastra
 - b. Zarządzanie security context
 - c. Używanie kont serwisowych do dostępu do API klastra
 - i. `kubectl create -f token.yaml`
 - ii. `kubectl config set-credentials my-sa-user --token=SA_TOKEN`
 - iii. `kubectl config set-cluster CLUSTER_NAME \`
`--server=CLUSTER_API_ENDPOINT \`
`--certificate-authority=PATH_TO_CA_FILE] \`
`[--insecure-skip-tls-verify=true]`
 - d. Tworzenie polityk RBAC

```
kubectl create clusterrolebinding my-sa-cluster-admin-binding \
  --clusterrole=cluster-admin \
  --serviceaccount=default:my-sa

kubectl create rolebinding my-sa-read-binding \
  --role=read-pods \
  --serviceaccount=default:my-sa \
  --namespace=default
```

- e. Konfiguracja ról klastra oraz RoleBindings

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

20. Logowanie, Monitorowanie oraz rozwiązywanie problemów z klastrem
 - a. Monitorowanie zasobów Kubernetes
 - i. **kubectl top pod**
 - ii. **kubectl top nodes**
 - b. Zapoznanie się z narzędziami pomocnymi w rozwiązywaniu problemów z klastrem
 - i. **kubectl describe**
 - ii. **kubectl events**
 - c. Rozwiązywanie problemów z aplikacjami
 - i. **kubectl get**
 - ii. **kubectl logs**
 - d. Rozwiązywanie problemów z węzłami klastra Kubernetes:
 - i. **sudo systemctl status kubelet**
 - ii. **sudo systemctl restart kubelet**
 - iii. **sudo openssl x509 -n /var/lib/kubelet/pki/kubelet.crt -text**
 - iv. **kubectl get pods -n kube-system**
 - e. Rozwiązywanie problemów z dostępem do aplikacji pracujących w ramach klastra:
 - i. **kubectl get endpointsslice**
 - ii. **kubectl describe svc**
21. Harbor jako repozytorium obrazów
 - a. Dlaczego warto używać Harbor do bezpiecznego przechowywania obrazów?
 - b. Omówienie różnic pomiędzy repozytoriami artefaktów
 - c. Konfiguracja i przesyłanie obrazów do Harbor
 - i. **y9zy99lc.c1.de1.container-registry.ovh.net**
22. Monitorowanie i diagnostyka
 - a. Podstawy monitorowania
 - b. Prometheus, Grafana i Kubernetes Dashboard
 - i. Instalacja

```
helm repo add prometheus-community \
https://prometheus-community.github.io/helm-charts

helm repo update

helm install prometheus-stack \
prometheus-community/kube-prometheus-stack \
--namespace=prometheus-stack \
--create-namespace

kubectl get pod -n prometheus-stack

kubectl -n prometheus-stack \
port-forward svc/prometheus-stack-grafana \
--address 0.0.0.0 3000:80 &

helm -n prometheus-stack show values \
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

```
prometheus-community/kube-prometheus-stack | grep -i \
adminpassword
```

ii. Omówienie serviceMonitor, podMonitor

23. Integracja wdrażania z GitOps

- Przegląd strategii GitOps
- Zasady i korzyści GitOps dla CI/CD
- Repozytoria
- Struktura repozytoriów gotowych na GitOps
- Konfigurowanie przepływu pracy GitOps
- Praktyczny przykład z wykorzystaniem ArgoCD

Instalacja:

```
kubectl create namespace argocd

kubectl config set-context --namespace=argocd --current

kubectl apply -n argocd -f \
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

Pobranie binarki na stację studenta:
curl -sSL -o argocd-linux-amd64 \
https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64

sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd

kubectl port-forward svc/argocd-server -n argocd 8080:443 &

ENTER

kubectl get secret argocd-initial-admin-secret -o \
jsonpath='{.data.password}' | base64 -d

argocd login localhost:8080 --username admin
```

Dostarczanie aplikacji

```
kubectl create ns gitops

kubectl config set-context --current --namespace=gitops

argocd app create guestbook --repo \
```

© 2025 Linux Polska Sp. z o.o. Wszelkie prawa zastrzeżone.

Niniejsze materiały szkoleniowe są chronione prawem autorskim i mogą być wykorzystywane wyłącznie przez uprawnionych użytkowników w celach szkoleniowych. Kopiowanie, rozpowszechnianie lub modyfikacja bez zgody autora jest zabroniona.

```
https://github.com/argoproj/argocd-example-apps.git \
--path guestbook \
--dest-server https://kubernetes.default.svc \
--dest-namespace gitops

argocd app get guestbook

argocd app sync guestbook

kubectl get po

argocd app delete guestbook
```